

Objective 1 – Conceptual Foundations:

HBase Architecture

HBase

For HBase, the HMaster manages the cluster by assigning regions to region servers, balancing the load, and monitoring overall health. Region servers are the worker nodes that handle client read and write requests, with each server managing one or more regions. Which are ranges of rows from a table. Data is then stored as HFiles in HDFS, allowing HBase to be scalable and fault-tolerant, since regions can be reassigned if a server fails. ZooKeeper coordinates the system by tracking the active region server and then communicating directly with it. HBase is a wide-column store that organizes data into tables, rows, and column families, where related columns are grouped and stored together on disk. HBase is good for real-time workload.

HBase Row Key Design

In HBase, data is stored and read based on the row key, so a poorly designed row key can cause serious performance problems. Using predictable keys like timestamps or sequential numbers creates hotspots where one server handles most of the work, while others sit idle. To avoid this, row keys should include a random prefix or salt to spread data evenly across regions. Additionally, columns that are frequently accessed together should be placed in the same column family, since each column family is stored separately on disk.

SQL vs NoSQL

Traditional SQL databases like MySQL, PostgreSQL, and Oracle are relational databases. They store data in tables with fixed schemas and support transactions, making it ideal for structured data and workloads that require consistency. NoSQL databases like HBase's are designed for flexibility and horizontal scalability, as it can handle large amounts of data by spreading it across many servers. HBASE is a wide-column database that provides fast access to massive datasets without requiring a fixed schema. SQL and NoSQL databases solve different problems, like SQL is best for structured and relational data. While NoSQL focuses on scalability and flexibility with big data.

Objective 2 – HBase Table Creation and Management:

```
ssh -L 9870:localhost:9870 -L 8088:localhost:8088 -L 8080:localhost:8080 -L  
18080:localhost:18080 -L 16010:localhost:16010 -L 8983:localhost:8983 -L 8443:localhost:8443  
meganpokal27@35.236.15.189
```

```
meganpokal — meganpokal27@dsc650bigdata: ~ — ssh -L 9870:localhost:9870 ...
...ssh meganpokal27@35.236.15.189 ...
...meganpokal27@35.236.15.189 +

Last login: Thu Jan 29 09:50:17 on ttys000

The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208050.
[Megans-MBP:~ meganpokal$ ssh -L 9870:localhost:9870 -L 8088:localhost:8088 -L 8080:localhost:8080 -L 18080:localhost:18080 -L 16010:localhost:16010 -L 8983:localhost:8983 -L 8443:localhost:8443 meganpokal27@35.236.15.189
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1020-gcp x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

Expanded Security Maintenance for Applications is not enabled.

39 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

4 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

Last login: Thu Jan 29 17:51:49 2026 from 76.227.225.11
meganpokal27@dsc650bigdata:~$
```

1. Environment Initialization

- Start by navigating to the required directory and initiating the Docker containers:

```
cd dsc650-infra/bellevue-bigdata/hadoop-hive-spark-hbase
```

```
docker-compose up -d
```

- Access the master container:

```
docker-compose exec master bash
```

```
meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bigdata/hadoop-hive-spark-hbase — ssh m...
...op-hive-spark-hbase — ssh meganpokal27@35.236.15.189 ...-L 8443:localhost:8443 meganpokal27@35.236.15.189 +

meganpokal27@dsc650bigdata:~/dsc650-infra/bellevue-bigdata$ ls
hadoop-hive-spark-hbase kafka hifi solr
meganpokal27@dsc650bigdata:~/dsc650-infra/bellevue-bigdata$ cd hadoop-hive-spark-hbase/
meganpokal27@dsc650bigdata:~/dsc650-infra/bellevue-bigdata/hadoop-hive-spark-hbase$ docker-compose up -d
Starting hadoop-hive-spark-hbase_hivemetastore_1 ...
hadoop-hive-spark-hbase_zoo2_1 is up-to-date
hadoop-hive-spark-hbase_zoo1_1 is up-to-date
Starting hadoop-hive-spark-hbase_hivemetastore_1 ... done
Starting hadoop-hive-spark-hbase_master_1 ... done
Starting hadoop-hive-spark-hbase_worker2_1 ... done
Starting hadoop-hive-spark-hbase_worker1_1 ... done
meganpokal27@dsc650bigdata:~/dsc650-infra/bellevue-bigdata/hadoop-hive-spark-hbase$ docker-compose exec master bash
bash-5.0#
```

2. Introduction to HBase

Enter the HBase interactive shell:

```
hbase shell
```

Starting up the HBase shell

```
meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bi...
...— ssh meganpokal27@35.236.15.189 ...3 meganpokal27@35.236.15.189 +

Starting hadoop-hive-spark-hbase_master_1 ... done
Starting hadoop-hive-spark-hbase_worker2_1 ... done
Starting hadoop-hive-spark-hbase_worker1_1 ... done
[meganpokal27@dsc650bigdata:~/dsc650-infra/bellevue-bigdata/hadoop-hive-spark-hbase]
$ docker-compose exec master bash
bash-5.0# hbase shell
2026-01-29 18:21:30,146 WARN [main] util.NativeCodeLoader: Unable to load native-
hadoop library for your platform... using builtin-java classes where applicable
[HBase Shell]
Use "help" to get list of supported commands.
Use "exit" to quit this interactive shell.
For Reference, please visit: http://hbase.apache.org/2.0/book.html#shell
Version 2.3.6, r7414579f2620fca6b75146c29ab2726fc4643ac9, Wed Jul 28 22:24:42 UTC
2021
Took 0.0012 seconds
hbase(main):001:0>
```

3. Table Creation and Management

Exercise 1: Create a table named 'students' with a column family 'details'.

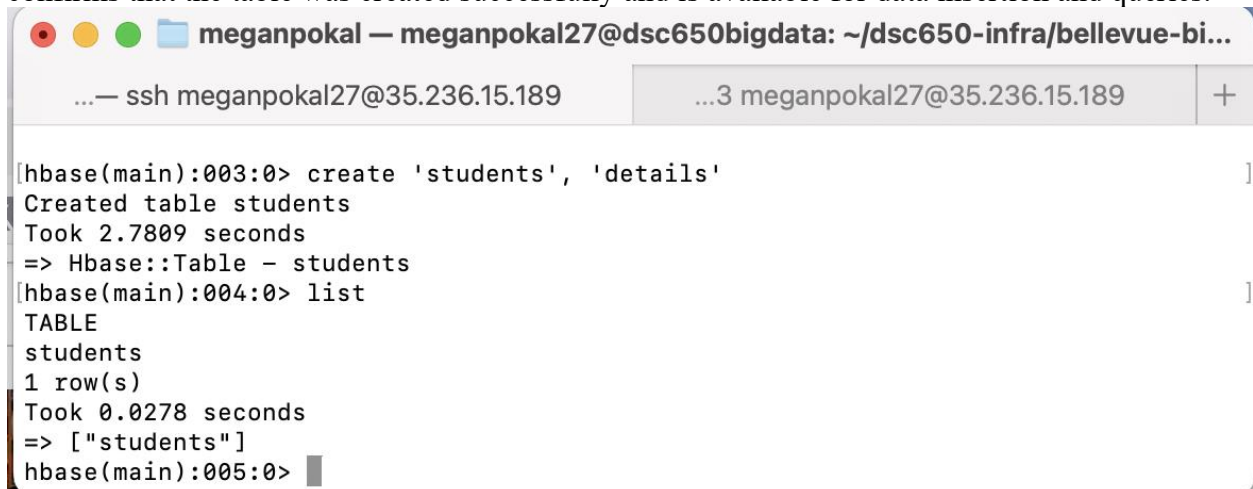
```
create 'students', 'details'
```

The output confirms that the students table was successfully created in HBase with a column family named details. The tables is now ready to store student information using row keys and column based data storage.

Exercise 2: Verify that the table has been created.

list

The list command displays all existing tables in HBase. Seeing the students table in the output confirms that the table was created successfully and is available for data insertion and queries.

A terminal window titled 'meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bi...' with tabs for 'ssh meganpokal27@35.236.15.189' and '...3 meganpokal27@35.236.15.189'. The terminal shows the following commands and output:

```
[hbase(main):003:0> create 'students', 'details'
Created table students
Took 2.7809 seconds
=> Hbase::Table - students
[hbase(main):004:0> list
TABLE
students
1 row(s)
Took 0.0278 seconds
=> ["students"]
hbase(main):005:0>
```

Objective 3 – HBase Data Manipulation

1. Data Manipulation in HBase

Exercise 3: Add data to the ‘students’ table. Let’s assume each student has a unique ID, a first name, and a last name.

```
put 'students', '1', 'details:firstName', 'John'
put 'students', '1', 'details:lastName', 'Doe'
```

The students first and last names are stored as key-value pairs under the details column family. This is how HBase stores data by row key and column.

A terminal window titled 'meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bi...' with tabs for 'ssh meganpokal27@35.236.15.189' and '...3 meganpokal27@35.236.15.189'. The terminal shows the following commands and output:

```
Took 0.0278 seconds
=> ["students"]
[hbase(main):005:0> put 'students', '1', 'details:firstName', 'John'
Took 0.2737 seconds
[hbase(main):006:0> put 'students', '1', 'details:lastName', 'Doe'
Took 0.0131 seconds
hbase(main):007:0>
```

Exercise 4: Query the data from the ‘students’ table to retrieve the details of the student with ID ‘1’.

```
get 'students', '1'
```

The output shows that the students first name and last name were successfully inserted into the students table and are stored under the details column family.

```
meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bi...
...— ssh meganpokal27@35.236.15.189    ...3 meganpokal27@35.236.15.189    +
Took 0.0131 seconds
[hbase(main):007:0> get 'students', '1'
COLUMN          CELL
details:firstName timestamp=2026-01-29T18:27:12.565, value=John
details:lastName  timestamp=2026-01-29T18:27:24.096, value=Doe
1 row(s)
Took 0.1040 seconds
hbase(main):008:0>
```

Objective 4 – Composite Row Keys with HBase

1. Advanced HBase Features: Composite Row Key

Exercise 5: Create a table named 'orders' to store data about customer orders. Assume each order is uniquely identified by a composite key formed by combining the customer ID and order date (in the format YYYYMMDD).

```
create 'orders', 'orderDetails'
```

It combined the customer ID and order data, allowing related orders to be stored together. Also, retrieving all orders for a specific customer.

```
meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bi...
...— ssh meganpokal27@35.236.15.189    ...3 meganpokal27@35.236.15.189    +
details:firstName timestamp=2026-01-29T18:27:12.565, value=John
details:lastName  timestamp=2026-01-29T18:27:24.096, value=Doe
1 row(s)
Took 0.1040 seconds
[hbase(main):008:0> create 'orders', 'orderDetails'
Created table orders
Took 2.1456 seconds
=> Hbase::Table - orders
hbase(main):009:0>
```

Exercise 6: Add sample data to the 'orders' table using the composite key:

```
put 'orders', '101:20230806', 'orderDetails:item', 'Laptop'
put 'orders', '102:20230806', 'orderDetails:item', 'Smartphone'
```


Each row key is a composite key made up of the customer ID and the order date in YYYYMMDD format.

```
meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bigdata/...
...ase — ssh meganpokal27@35.236.15.189 ...8443 meganpokal27@35.236.15.189 +
Created table orders
Took 2.1456 seconds
=> Hbase::Table - orders
hbase(main):009:0> put 'orders', '101:20230806', 'orderDetails:item', 'Laptop'
Took 0.0280 seconds
hbase(main):010:0> put 'orders', '102:20230806', 'orderDetails:item', 'Smartphone'
Took 0.0107 seconds
hbase(main):011:0> █
```

Exercise 7: Query the ‘orders’ table to retrieve details of all orders placed by the customer with ID ‘101’.

```
scan 'orders', {STARTROW => '101:', ENDROW => '101:~'}
```

This command will scan rows starting from ‘101:’ to before ‘101:~’ (tilde ‘~’ is the next ASCII character after colon ‘:’).

The scan has filtered and returned all orders associated with customer ID 101.

```
meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bigdata/...
...ase — ssh meganpokal27@35.236.15.189 ...8443 meganpokal27@35.236.15.189 +
Took 0.0280 seconds
hbase(main):010:0> put 'orders', '102:20230806', 'orderDetails:item', 'Smartphone'
Took 0.0107 seconds
hbase(main):011:0> scan 'orders', {STARTROW => '101:', ENDROW => '101:~'}
ROW COLUMN+CELL
101:20230806 column=orderDetails:item, timestamp=2026-01-29T18:30:59.371, value=Laptop
1 row(s)
Took 0.0292 seconds
hbase(main):012:0> █
```

Objective 5 – Data Generation in HBase

1. Data Generation for HBase

Exercise 8: Generate random data for the ‘students’ table.

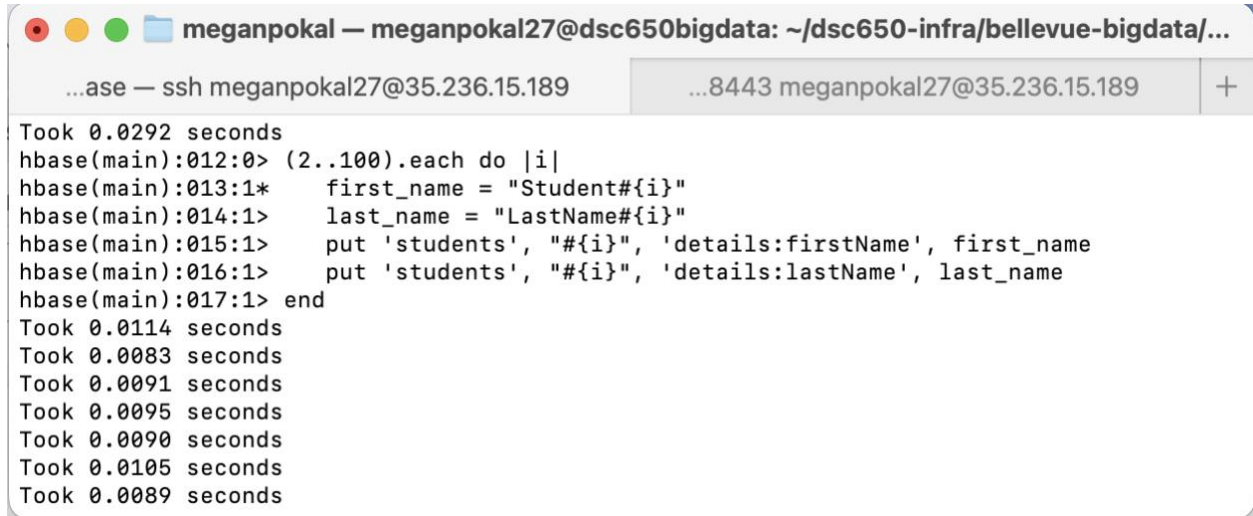
```
(2..100).each do |i|
```

```
  first_name = "Student#{i}"
```

```
  last_name = "LastName#{i}"
```

```
put 'students', "#{i}", 'details:firstName', first_name
put 'students', "#{i}", 'details:lastName', last_name
end
```

This creates a loop through student IDs 2 through 100 and generates sample data for each row in the students table. Inserting it into the details:firstName and details:lastName columns.



```
meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bigdata/...
...ase — ssh meganpokal27@35.236.15.189    ...8443 meganpokal27@35.236.15.189    +
Took 0.0292 seconds
hbase(main):012:0> (2..100).each do |i|
hbase(main):013:1*   first_name = "Student#{i}"
hbase(main):014:1>   last_name = "LastName#{i}"
hbase(main):015:1>   put 'students', "#{i}", 'details:firstName', first_name
hbase(main):016:1>   put 'students', "#{i}", 'details:lastName', last_name
hbase(main):017:1> end
Took 0.0114 seconds
Took 0.0083 seconds
Took 0.0091 seconds
Took 0.0095 seconds
Took 0.0090 seconds
Took 0.0105 seconds
Took 0.0089 seconds
```

Exercise 9: Scan the 'students' table to verify data insertion.

scan 'students'

This makes sure that the students table was successful and each row represents a unique student ID and displays the correct columns under details, like firstname, lastname and middle name.

```
meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bigdata/...
...ase — ssh meganpokal27@35.236.15.189 ...8443 meganpokal27@35.236.15.189 +
Took 0.0046 seconds
Took 0.0044 seconds
Took 0.0046 seconds
=> 2..100
[hbase(main):018:0> scan 'students' ]
ROW COLUMN+CELL
1 column=details:firstName, timestamp=2026-01-29T18:27:12.565, value=John
1 column=details:lastName, timestamp=2026-01-29T18:27:24.096, value=Doe
10 column=details:firstName, timestamp=2026-01-29T18:35:14.694, value=Student10
10 column=details:lastName, timestamp=2026-01-29T18:35:14.703, value=LastName10
100 column=details:firstName, timestamp=2026-01-29T18:35:15.794, value=Student100
```



```
meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bigdata/ha...
...base — ssh meganpokal27@35.236.15.189      ...t:8443 meganpokal27@35.236.15.189      +
89      column=details:firstName, timestamp=2026-01-29T18:35:15.686, val
ue=Student89
89      column=details:lastName, timestamp=2026-01-29T18:35:15.692, valu
e=LastName89
9      column=details:firstName, timestamp=2026-01-29T18:35:14.676, val
ue=Student9
9      column=details:lastName, timestamp=2026-01-29T18:35:14.685, valu
e=LastName9
90     column=details:firstName, timestamp=2026-01-29T18:35:15.699, val
ue=Student90
90     column=details:lastName, timestamp=2026-01-29T18:35:15.704, valu
e=LastName90
91     column=details:firstName, timestamp=2026-01-29T18:35:15.709, val
ue=Student91
91     column=details:lastName, timestamp=2026-01-29T18:35:15.714, valu
e=LastName91
92     column=details:firstName, timestamp=2026-01-29T18:35:15.718, val
ue=Student92
92     column=details:lastName, timestamp=2026-01-29T18:35:15.723, valu
e=LastName92
93     column=details:firstName, timestamp=2026-01-29T18:35:15.727, val
ue=Student93
93     column=details:lastName, timestamp=2026-01-29T18:35:15.731, valu
e=LastName93
94     column=details:firstName, timestamp=2026-01-29T18:35:15.736, val
ue=Student94
94     column=details:lastName, timestamp=2026-01-29T18:35:15.741, valu
e=LastName94
95     column=details:firstName, timestamp=2026-01-29T18:35:15.745, val
ue=Student95
95     column=details:lastName, timestamp=2026-01-29T18:35:15.750, valu
e=LastName95
96     column=details:firstName, timestamp=2026-01-29T18:35:15.755, val
ue=Student96
96     column=details:lastName, timestamp=2026-01-29T18:35:15.759, valu
e=LastName96
97     column=details:firstName, timestamp=2026-01-29T18:35:15.764, val
ue=Student97
97     column=details:lastName, timestamp=2026-01-29T18:35:15.769, valu
e=LastName97
98     column=details:firstName, timestamp=2026-01-29T18:35:15.774, val
ue=Student98
98     column=details:lastName, timestamp=2026-01-29T18:35:15.779, valu
e=LastName98
99     column=details:firstName, timestamp=2026-01-29T18:35:15.784, val
ue=Student99
99     column=details:lastName, timestamp=2026-01-29T18:35:15.789, valu
e=LastName99
100 row(s)
Took 0.2065 seconds
hbase(main):019:0>
```

Objective 6 – Advanced Data Manipulation in HBase

Exercise 10: HBase Data Manipulation

1. Update First Names:

- For students with IDs from 2 to 50, change the first name prefix from Student to Scholar. For instance, Student3 should become Scholar3.

```
(2..50).each do |i|
  put 'students', "row#{i}", 'details:first_name', "Scholar#{i}"
end
```

This update replaces the existing first name values for students with IDs 2 - 50 by overwriting the column using the put command. HBase updates data by writing a new version of the cell

```
meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bigdata/ha...
...base — ssh meganpokal27@35.236.15.189    ...t:8443 meganpokal27@35.236.15.189    +
hbase(main):017:0> (2..50).each do |i|
hbase(main):018:1*   put 'students', "row#{i}", 'details:first_name', "Scholar#{i}"
hbase(main):019:1> end
Took 0.0036 seconds
Took 0.0033 seconds
Took 0.0028 seconds
Took 0.0023 seconds
Took 0.0027 seconds
Took 0.0029 seconds
Took 0.0028 seconds
Took 0.0026 seconds
Took 0.0031 seconds
Took 0.0027 seconds
Took 0.0024 seconds
Took 0.0031 seconds
```

```
get 'students', '3'
```

```
meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bigdata/ha...
...base — ssh meganpokal27@35.236.15.189    ...t:8443 meganpokal27@35.236.15.189    +
hbase(main):046:0> get 'students', '3'
COLUMN          CELL
  details:firstName  timestamp=2026-01-29T19:42:46.229, value=Scholar3
  details:lastName   timestamp=2026-01-29T19:29:02.575, value=LastName3
1 row(s)
Took 0.0066 seconds
hbase(main):047:0>
```

Add a Middle Name:

- For students with IDs from 51 to 75, add a middle name column under the details column family. The middle name should follow the pattern MidName#{i}.

```
(51..75).each do |i|
  put 'students', "row#{i}", 'details:middle_name', "MidName###{i}"
end
```

This will add a new column, `middle_name`, under the `details` column family for students with IDs 51-75. HBase will allow us to create a column without altering the table schema.

```
meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bigdata/ha...
...base — ssh meganpokal27@35.236.15.189    ...t:8443 meganpokal27@35.236.15.189    +

Took 0.0330 seconds
hbase(main):024:0> (51..75).each do |i|
hbase(main):025:1*   put 'students', "#{i}", 'details:middle_name', "MidName##{i}"
[hbase(main):026:1> end
Took 0.0045 seconds
Took 0.0044 seconds
Took 0.0044 seconds
Took 0.0040 seconds
Took 0.0040 seconds
Took 0.0037 seconds
Took 0.0035 seconds
Took 0.0034 seconds
Took 0.0027 seconds
Took 0.0034 seconds
Took 0.0027 seconds
Took 0.0027 seconds
Took 0.0033 seconds
Took 0.0027 seconds
Took 0.0034 seconds
Took 0.0032 seconds
Took 0.0027 seconds
Took 0.0027 seconds
Took 0.0026 seconds
Took 0.0031 seconds
Took 0.0031 seconds
Took 0.0035 seconds
Took 0.0028 seconds
Took 0.0025 seconds
Took 0.0029 seconds
=> 51..75
hbase(main):027:0> █
```

get 'students', '60'

```
meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bigdata/ha...
...base — ssh meganpokal27@35.236.15.189    ...t:8443 meganpokal27@35.236.15.189    +

Took 0.0018 seconds
=> 51..75
[hbase(main):050:0> get 'students', '60'
COLUMN                                CELL
  details:firstName                    timestamp=2026-01-29T19:29:03.167, value=Student60
  details:lastName                     timestamp=2026-01-29T19:29:03.172, value=LastName60
  details:middle_name                  timestamp=2026-01-29T19:45:32.801, value=MidName#60
1 row(s)
Took 0.0058 seconds
hbase(main):051:0> █
```

Modify Last Names:

- For students with IDs from 76 to 100, append `_Modified` to the last name. So, `LastName76` should be updated to `LastName76_Modified`.

```
(76..100).each do |i|  
  put 'students', "#{i}", 'details:last_name', "LastName#{i}_Modified"  
end
```

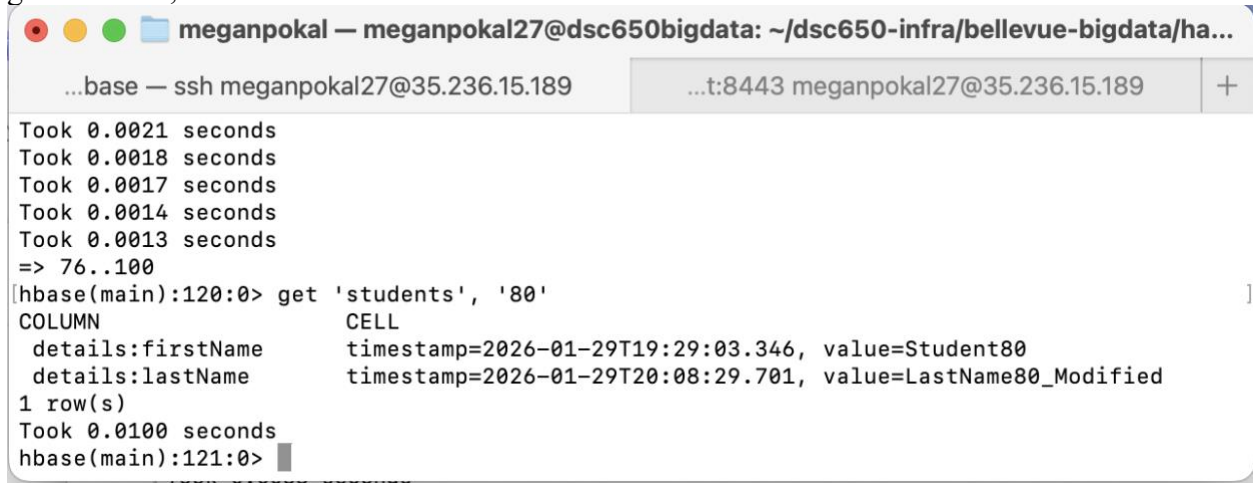


A terminal window titled "meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bigdata/ha..." is shown. The terminal has tabs for "...base — ssh meganpokal27@35.236.15.189" and "...t:8443 meganpokal27@35.236.15.189". The command history shows the execution of the HBase command from the previous block. The output shows the command being executed at hbase(main):117:0*, followed by the command at hbase(main):118:1*, and then the command at hbase(main):119:1> end. The output then shows a series of "Took" messages indicating the time taken for each of the 25 iterations (from ID 76 to 100). The times range from 0.0013 to 0.0028 seconds. The final output is "=> 76..100".

```
meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bigdata/ha...  
...base — ssh meganpokal27@35.236.15.189    ...t:8443 meganpokal27@35.236.15.189  +  
hbase(main):117:0* (76..100).each do |i|  
hbase(main):118:1*   put 'students', "#{i}", 'details:lastName', "LastName#{i}_Modified"  
hbase(main):119:1> end  
Took 0.0028 seconds  
Took 0.0020 seconds  
Took 0.0020 seconds  
Took 0.0018 seconds  
Took 0.0019 seconds  
Took 0.0016 seconds  
Took 0.0014 seconds  
Took 0.0025 seconds  
Took 0.0018 seconds  
Took 0.0019 seconds  
Took 0.0022 seconds  
Took 0.0022 seconds  
Took 0.0020 seconds  
Took 0.0022 seconds  
Took 0.0020 seconds  
Took 0.0020 seconds  
Took 0.0020 seconds  
Took 0.0023 seconds  
Took 0.0024 seconds  
Took 0.0022 seconds  
Took 0.0021 seconds  
Took 0.0018 seconds  
Took 0.0017 seconds  
Took 0.0014 seconds  
Took 0.0013 seconds  
=> 76..100
```

The last name for each student from ID 76 to 100 was updated by appending `_Modified`. This was done by overwriting the existing `last_name` column values using `put`.

```
get 'students', '80'
```



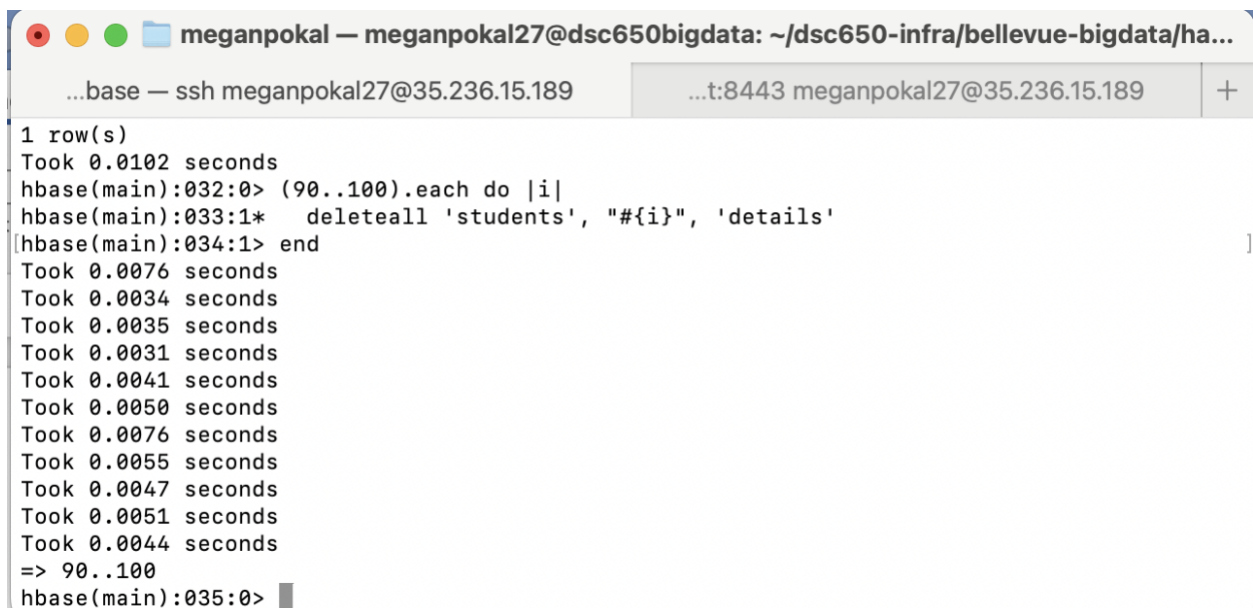
```
meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bigdata/ha...
...base — ssh meganpokal27@35.236.15.189    ...t:8443 meganpokal27@35.236.15.189    +
Took 0.0021 seconds
Took 0.0018 seconds
Took 0.0017 seconds
Took 0.0014 seconds
Took 0.0013 seconds
=> 76..100
[hbase(main):120:0> get 'students', '80'
COLUMN                                CELL
  details:firstName                    timestamp=2026-01-29T19:29:03.346, value=Student80
  details:lastName                     timestamp=2026-01-29T20:08:29.701, value=LastName80_Modified
1 row(s)
Took 0.0100 seconds
hbase(main):121:0>
```

Bulk Delete:

- Delete all the details for students with IDs from 90 to 100.

```
(90..100).each do |i|
  deleteall 'students', "#{i}", 'details'
end
```

The bulk delete removed all columns under the details column family for students with IDs 90-100. The deleteall command permanently removes the specified column family data from each row.



```
meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bigdata/ha...
...base — ssh meganpokal27@35.236.15.189    ...t:8443 meganpokal27@35.236.15.189    +
1 row(s)
Took 0.0102 seconds
hbase(main):032:0> (90..100).each do |i|
hbase(main):033:1*   deleteall 'students', "#{i}", 'details'
[hbase(main):034:1> end
Took 0.0076 seconds
Took 0.0034 seconds
Took 0.0035 seconds
Took 0.0031 seconds
Took 0.0041 seconds
Took 0.0050 seconds
Took 0.0076 seconds
Took 0.0055 seconds
Took 0.0047 seconds
Took 0.0051 seconds
Took 0.0044 seconds
=> 90..100
hbase(main):035:0>
```



```
get 'students', '90'
```

Run get, row 90 to make sure that it has been successfully deleted.

A terminal window titled 'meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bigdata/ha...' shows a session with 'hbase(main):083:0'. The user enters 'get 'students', '90'', and the output shows 'COLUMN' and 'CELL' headers, followed by '0 row(s)' and 'Took 0.0037 seconds'. The prompt then changes to 'hbase(main):084:0>'.

```
meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bigdata/ha...
...base — ssh meganpokal27@35.236.15.189    ...t:8443 meganpokal27@35.236.15.189    +
hbase(main):083:0> get 'students', '90'
COLUMN                                CELL
0 row(s)
Took 0.0037 seconds
hbase(main):084:0>
```

Data Retrieval:

- After all modifications, retrieve and display the details for students with IDs 40, 60, 80, and 90 to verify changes.

```
get 'students', '40'
```

```
get 'students', '60'
```

```
get 'students', '80'
```

```
get 'students', '90'
```

This will show that all the different commands have been completed correctly. Showing firtName updated, firstName unchanged, middle_name, lastName modified, and then the last part has been deleted and there are no rows.


```
meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bigdata/ha...
...base — ssh meganpokal27@35.236.15.189    ...t:8443 meganpokal27@35.236.15.189    +

Took 0.0100 seconds
hbase(main):121:0> get 'students', '40'
COLUMN      CELL
  details:firstName    timestamp=2026-01-29T19:54:53.577, value=Scholar40
  details:lastName     timestamp=2026-01-29T19:29:02.997, value=LastName40
1 row(s)
Took 0.0056 seconds
hbase(main):122:0> get 'students', '60'
COLUMN      CELL
  details:firstName    timestamp=2026-01-29T19:29:03.167, value=Student60
  details:lastName     timestamp=2026-01-29T19:29:03.172, value=LastName60
  details:middle_name  timestamp=2026-01-29T19:45:32.801, value=MidName#60
1 row(s)
Took 0.0053 seconds
hbase(main):123:0> get 'students', '80'
COLUMN      CELL
  details:firstName    timestamp=2026-01-29T19:29:03.346, value=Student80
  details:lastName     timestamp=2026-01-29T20:08:29.701, value=LastName80_Modified
1 row(s)
Took 0.0047 seconds
hbase(main):124:0> get 'students', '90'
COLUMN      CELL
  details:lastName     timestamp=2026-01-29T20:08:29.724, value=LastName90_Modified
1 row(s)
Took 0.0040 seconds
hbase(main):125:0> █
```

Final Table Scan

```
scan 'students'
```

The final scan confirms that all updates, additions, and deletions were applied correctly. The table reflects all the changes that I have implemented.

```
meganpokal — meghanpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bigdata/ha...
...base — ssh meghanpokal27@35.236.15.189    ...t:8443 meghanpokal27@35.236.15.189    +

1 row(s)
Took 0.0040 seconds
[hbase(main):125:0> scan 'students' ]
ROW      COLUMN+CELL
 1      column=details:firstName, timestamp=2026-01-29T19:27:30.577, value=
        =John
 1      column=details:lastName, timestamp=2026-01-29T19:27:41.532, value=
        =Doe
10      column=details:firstName, timestamp=2026-01-29T19:54:53.489, value=
        =Scholar10
10      column=details:lastName, timestamp=2026-01-29T19:29:02.664, value=
        =LastName10
100     column=details:firstName, timestamp=2026-01-29T19:29:03.497, value=
        =Student100
100     column=details:lastName, timestamp=2026-01-29T20:08:29.747, value=
        =LastName100_Modified
11      column=details:firstName, timestamp=2026-01-29T19:54:53.493, value=
        =Scholar11
11      column=details:lastName, timestamp=2026-01-29T19:29:02.677, value=
        =LastName11
12      column=details:firstName, timestamp=2026-01-29T19:54:53.497, value=
        =Scholar12
12      column=details:lastName, timestamp=2026-01-29T19:29:02.691, value=
        =LastName12
13      column=details:firstName, timestamp=2026-01-29T19:54:53.499, value=
        =Scholar13
13      column=details:lastName, timestamp=2026-01-29T19:29:02.702, value=
        =LastName13
14      column=details:firstName, timestamp=2026-01-29T19:54:53.503, value=
```

meganpokal — megalpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bigdata/ha...

...base — ssh megalpokal27@35.236.15.189

...t:8443 megalpokal27@35.236.15.189

+

row43	column=details:first_name, timestamp=2026-01-29T19:29:29.275, value=Scholar43
row44	column=details:first_name, timestamp=2026-01-29T19:29:29.279, value=Scholar44
row45	column=details:first_name, timestamp=2026-01-29T19:29:29.282, value=Scholar45
row46	column=details:first_name, timestamp=2026-01-29T19:29:29.286, value=Scholar46
row47	column=details:first_name, timestamp=2026-01-29T19:29:29.290, value=Scholar47
row48	column=details:first_name, timestamp=2026-01-29T19:29:29.293, value=Scholar48
row49	column=details:first_name, timestamp=2026-01-29T19:29:29.297, value=Scholar49
row5	column=details:first_name, timestamp=2026-01-29T19:29:29.133, value=Scholar5
row50	column=details:first_name, timestamp=2026-01-29T19:29:29.301, value=Scholar50
row6	column=details:first_name, timestamp=2026-01-29T19:29:29.136, value=Scholar6
row7	column=details:first_name, timestamp=2026-01-29T19:29:29.139, value=Scholar7
row8	column=details:first_name, timestamp=2026-01-29T19:29:29.142, value=Scholar8
row9	column=details:first_name, timestamp=2026-01-29T19:29:29.146, value=Scholar9

149 row(s)

Took 0.0912 seconds

hbase(main):126:0>